

SCHOOL OF TECHNOLOGY



DATABASE MANAGEMENT SYSTEMS
PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python

PROJECT TITLE

**CAMPUS PLACEMENT AND RECRUITMENT DRIVE
MANAGEMENT SYSTEM**

Program	B.Tech
Section	AIML RHINOS
Faculty	Prof. Sahithi
Date of Submission	10-10-2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102074	Farhan Ansari	7305314185	ansarisj12@gmail.com

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, Prof. Sahithi, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled "**Campus Placement and Recruitment Drive Management System**", submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Signature

Date : 10-10-2026

Place : Hyderabad

Name: Farhan Ansari

Roll Number: 25WU0102074

Program: B.Tech AIML

TABLE OF CONTENTS

Contents	Page No.
LIST OF TABLES	iv
LIST OF FIGURES	v
LIST OF ACRONYMS	vi
ABSTRACT	vii
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	1
1.1 Problem Statement	1
1.2 Objectives	1
1.3 Scope and Assumptions	2
1.4 Proposed Solution Strategy	2
1.5 Functional Requirements	3
1.6 Entity-Relationship Diagram (ERD)	5
1.7 Entity and Relationship Summary	6
2. LOGICAL DATABASE DESIGN	7
2.1 Relational Schema	7
2.2 Keys and Integrity Constraints	7
2.3 Functional Dependencies	8
2.4 Normalization up to Third Normal Form (3NF)	9
2.5 Data Dictionary	9
2.6 Implementation of Logical Design: DDL and Sample Data	11
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	13
3.1 Application Overview	13
3.2 Development Environment and Database Connectivity	13
3.3 Application Modules and Database Mapping	14
3.4 CRUD Operations	15
3.5 Business Rules, Validation and Error Handling	16
3.6 Reports and Database Operations	17
3.7 Application Screens	18
3.8 Testing and Results	20
4. CONCLUSION	22
4.1 Limitations and Future Enhancements	22
5. REFERENCES	23
6. APPENDICES	26
Appendix A – Complete Database Script	26
Appendix B – Application Source Code	28
Appendix C – Additional Test Data and Outputs	29

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	2
Table 2	Proposed Solution Strategy	2
Table 3	Functional Requirements	3
Table 4	Entity and Relationship Summary	6
Table 5	Relational Schema	7
Table 6	Keys and Integrity Constraints	7
Table 7	Normalization Status up to 3NF	9
Table 8	Data Dictionary	9
Table 9	Database Implementation Environment	11
Table 10	Application Development Environment	13
Table 11	Application Module-to-Database Mapping	14
Table 12	CRUD Operations	15
Table 13	Business Rules, Validation and Error Handling	16
Table 14	Reports and Database Operations	17
Table 15	Test Cases	20

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	5
Figure 2	Executive Dashboard	18
Figure 3	Student Registration Form	18
Figure 4	Student Directory with Search and Filters	18
Figure 5	Application Submission with Live Eligibility Check	19
Figure 6	Interview Result Update Triggering Placement	19
Figure 7	Five-Table Master Report and placement_summary View	19

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
AIML	Artificial Intelligence and Machine Learning
BCNF	Boyce–Codd Normal Form
CGPA	Cumulative Grade Point Average
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FK	Foreign Key
FR	Functional Requirement
GUI	Graphical User Interface
HR	Human Resources
IDE	Integrated Development Environment
KPI	Key Performance Indicator
LPA	Lakhs Per Annum
PBL	Project-Based Learning
PK	Primary Key
SQL	Structured Query Language
UI	User Interface
UQ	Unique Key

ABSTRACT

Campus placement involves many connected activities, such as maintaining student profiles, recording company and job details, checking eligibility, receiving applications and tracking interview rounds. When these records are kept in spreadsheets and separate documents, they become duplicated, inconsistent and slow to search, and ineligible students may be considered by mistake. A database-based solution is therefore needed to keep placement data in one organized place. The objective of this project is to develop a centralized relational database application for the placement cell. An entity-relationship model was converted into a logical design of five relations, namely students, companies, job postings, applications and interviews, which satisfy third normal form and are protected by key, unique, check and default constraints in MySQL, together with a trigger and a view. A Python web application built with Streamlit connects to the database using parameterized queries. It provides create, read, update and delete operations for every entity, a live eligibility check, duplicate-application detection, an analytics dashboard, multi-table join reports and a read-only SQL explorer. When an interview result is marked as selected, the student is marked as placed automatically. The completed system shows how a well-designed schema supports consistent, easily retrievable placement data.

KEYWORDS

Keywords: Relational Database, ER Diagram, Normalization, MySQL, Streamlit, Campus Placement

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

In many institutions, campus placement information is managed using spreadsheets, documents or separate records maintained by different people. As the number of students, companies and job openings grows, this approach makes it difficult to manage large numbers of students, leads to duplicate or incorrect application records, and makes it hard to check whether a student is eligible for a job. Selection rounds, interview results and final outcomes are tracked manually, and generating a report for the placement cell is time-consuming. Manual screening can also result in ineligible students being considered. The placement administrator, the students and the recruiting companies are all affected by these limitations. Placement systems reported in the literature likewise aim to replace paperwork with a student database and to ensure that only eligible students get a chance [1], [2].

A database management system keeps data in a single structured store, reduces redundancy and enforces integrity rules [3]–[5]. There is therefore a need for a centralized database application that records students, companies, job postings, applications and interview rounds in related tables, and lets the placement cell enter, update and retrieve this information quickly and consistently.

1.2 Objectives

- Design an entity-relationship model and a relational schema in third normal form for students, companies, job postings, applications and interviews.
- Implement the schema in MySQL with primary key, foreign key, unique, check and default constraints, a trigger that marks selected students as placed, and a view that summarizes placements.
- Develop a Python web application using Streamlit that connects to MySQL and performs create, read, update and delete operations on all five entities using parameterized queries.
- Automate the eligibility check, duplicate-application detection and placement-status update, and generate dashboard analytics and multi-table join reports.
- Validate the application with positive and negative test cases covering data entry, constraint violations, the trigger and report display.

1.3 Scope and Assumptions

Table 1 defines what the implemented system covers and the assumptions made. The assumption on CHECK constraints follows the MySQL documentation, which states that such constraints are enforced from version 8.0.16 [6].

Table 1. Scope and Assumptions

Item	Description
In Scope	Student records: ID, name, email, phone, department, CGPA and placement status. Company records: name, HR email, industry and website. Job postings: title, package in LPA, minimum CGPA and deadline. Applications of students to postings, and interview rounds with date, interviewer, round number and result. Create, read, update and delete for all entities, with search, filters and CSV export. Eligibility check, duplicate-application detection and automatic placement status. Dashboard, five-table join report, placement_summary view and a read-only SQL explorer.
Out of Scope / Exclusions	User login and role-based access for administrators, students and recruiters. Blocking of applications from students below the minimum CGPA; the check is shown as a warning only. Email notifications, resume upload and online tests. Separate department, skill, academic-record and offer tables from the Review-I conceptual model; the needed details are held as attributes of the five implemented tables.
Assumptions	The portal is operated by a placement cell administrator on a machine with a local MySQL server. Each student has a unique ID and email; each company has a unique HR email. CGPA is on a 0 to 10 scale. The database <code>campus_placement</code> exists before the schema script is run. MySQL 8.0.16 or later is used so that CHECK constraints are enforced [6].

1.4 Proposed Solution Strategy

The proposed solution is a relational database with a Python web front end. The placement administrator registers students and companies, posts jobs, submits each student's application with a live eligibility message, schedules interview rounds and records their results. The database enforces keys and constraints and updates the student's placement status through a trigger, while the application supplies lookup lists, input validation, analytics and reports. Table 2 summarizes the strategy.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Placement cell administrator, who operates the web portal. Student and recruiter information is maintained by the administrator on their behalf.

Solution Element	Project-Specific Description
Major Application Modules	Executive dashboard; students directory; partner companies; job postings and drives; applications pipeline; interview management; advanced reports and views; database diagnostics and demo data.
Proposed Workflow	Register student and company → post job → submit application with eligibility check → schedule interview rounds → record results (student marked Placed on selection) → review dashboard and reports.
Important Business Rules	Unique student ID, student email and company HR email; CGPA between 0 and 10; a posting belongs to an existing company; an application refers to an existing student and posting and is not duplicated; an interview refers to an existing application; a Selected result marks the student as Placed; deleting a parent record removes its dependent records.
Expected Reports / Outputs	Dashboard KPIs and charts; student, company, job, application and interview lists with filters; five-table master report; placement_summary view; CSV exports; preset analytical SQL queries.

The Review-I conceptual model identified thirteen entities, including department, academic record, skill, placement drive, eligibility criteria, selection round, result and offer. For the working prototype these were consolidated into five relations: department and CGPA are stored in `students`; the minimum CGPA and deadline of a drive are stored in `job_postings`; selection rounds and their results are stored in `interviews`; and the final outcome is reflected by a Selected result and the student's placement status, which is maintained by a trigger.

1.5 Functional Requirements

Table 3 lists the functional requirements. Requirements FR-01 to FR-12 are implemented in the current application; FR-13 is planned and is discussed in Section 4.1.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	The administrator shall be able to register a student with ID, name, email, phone, department, CGPA and status.	High – Implemented
FR-02	The system shall display students and allow search by ID, name or email and filtering by department, status and minimum CGPA, with CSV export.	High – Implemented
FR-03	The administrator shall be able to edit and delete students.	Medium – Implemented
FR-04	The administrator shall be able to add, search, edit and delete companies, and see the number of jobs each has posted.	High – Implemented
FR-05	The administrator shall be able to post, filter, edit and delete job postings of an existing company.	High – Implemented
FR-06	The administrator shall be able to submit an application of a student for a job, with a live eligibility message and duplicate detection.	High – Implemented
FR-07	The system shall display applications, filter them by status, update their status and allow withdrawal.	Medium – Implemented

Req. ID	Functional Requirement	Remarks / Priority
FR-08	The administrator shall be able to schedule an interview round with date, time, interviewer, round number and result.	High – Implemented
FR-09	The administrator shall be able to update an interview result or cancel an interview; a Selected result shall mark the student as Placed.	High – Implemented
FR-10	The system shall present an executive dashboard with KPIs and charts.	Medium – Implemented
FR-11	The system shall provide a five-table master report, the placement_summary view, CSV export and a read-only SQL explorer.	High – Implemented
FR-12	The system shall show database connection status and load demo data.	Low – Implemented
FR-13	The system shall block applications from students whose CGPA is below the posting's minimum or after its deadline.	High – Planned

1.6 Entity-Relationship Diagram (ERD)

Figure 1 shows the final ER diagram of the system, drawn using the entity-relationship approach [7]. It has five entities and four one-to-many relationships. The many-to-many association between students and job postings is resolved by the applications entity,

which holds the foreign keys of both. The diagram is consistent with the relational schema and DDL presented in Chapter 2.

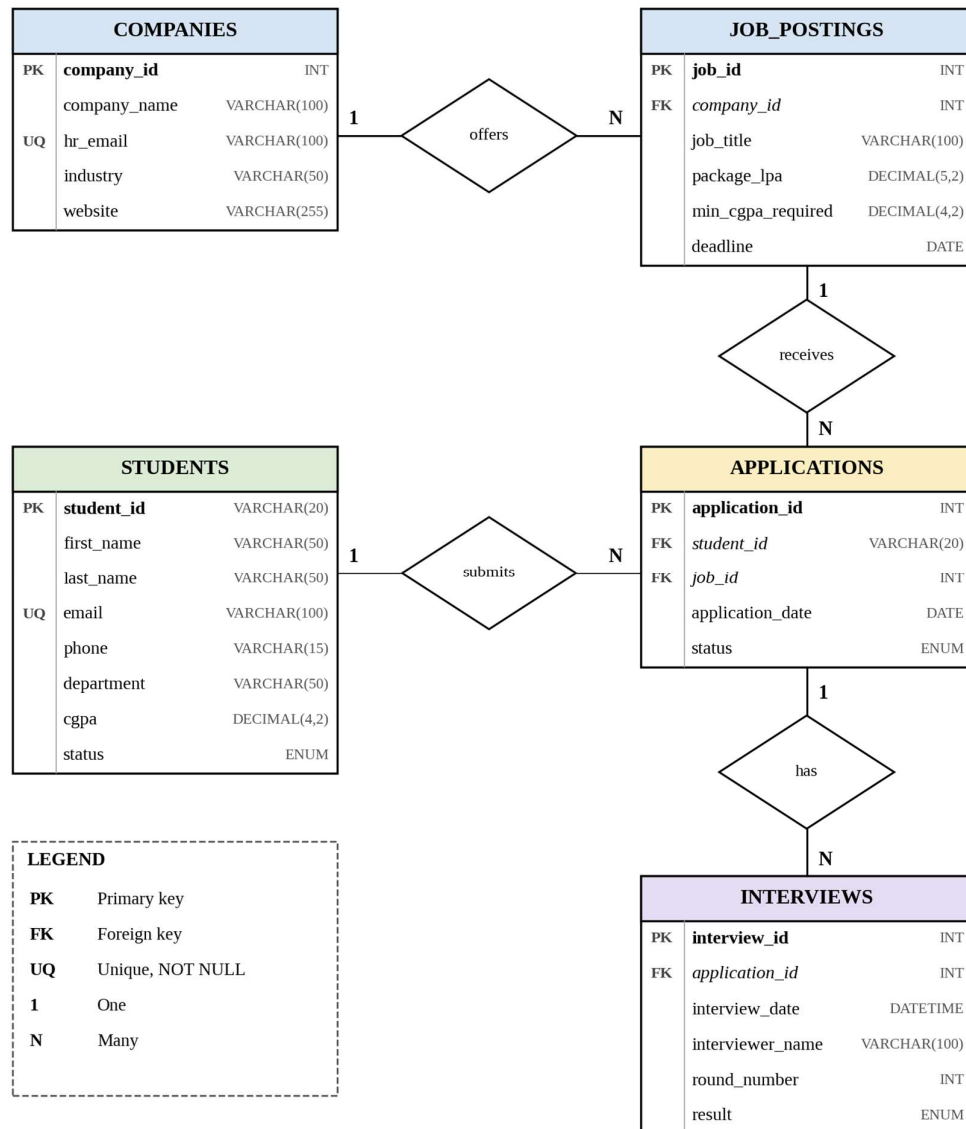


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 summarizes the entities and relationships shown in Figure 1.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
STUDENTS	student_id (PK), first_name, last_name, email (UQ), phone, department, cgpa, status	Stores the profile and placement status of each student.

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
COMPANIES	company_id (PK), company_name, hr_email (UQ), industry, website	Stores the recruiting companies.
JOB_POSTINGS	job_id (PK), company_id (FK), job_title, package_lpa, min_cgpa_required, deadline	Stores each opening and its eligibility cut-off.
APPLICATIONS	application_id (PK), student_id (FK), job_id (FK), application_date, status	Links a student to a job posting.
INTERVIEWS	interview_id (PK), application_id (FK), interview_date, interviewer_name, round_number, result	Tracks interview rounds and results of an application.
offers (Companies – Job_Postings)	1 : N	One company can offer many postings; each posting belongs to one company.
receives (Job_Postings – Applications)	1 : N	One posting can receive many applications.
submits (Students – Applications)	1 : N	One student can submit many applications.
has (Applications – Interviews)	1 : N	One application can have many interview rounds.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER diagram was converted to relations by mapping each entity to a table and placing the primary key of the "one" side as a foreign key on the "many" side of each one-to-many relationship, following the relational model [8] and the standard ER-to-relational mapping rules [3], [9]. Table 5 lists the five relations and the view built on them, and the schema is written in compact notation below it.

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	students	Student profile, CGPA and placement status
2	companies	Recruiting companies and HR contact
3	job_postings	Job openings of companies with package and eligibility cut-off
4	applications	Applications of students to job postings
5	interviews	Interview rounds and results for applications
6	placement_summary (view)	Selected candidates with department, company and package

```
students(student_id PK, first_name, last_name, email UQ, phone,  
         department, cgpa, status)  
companies(company_id PK, company_name, hr_email UQ, industry, website)  
job_postings(job_id PK, company_id FK, job_title, package_lpa,  
            min_cgpa_required, deadline)  
applications(application_id PK, student_id FK, job_id FK,  
            application_date, status)  
interviews(interview_id PK, application_id FK, interview_date,  
            interviewer_name, round_number, result)  
placement_summary [view] (first_name, last_name, department,  
                           company_name, package_lpa)
```

2.2 Keys and Integrity Constraints

Table 6 lists the keys and constraints implemented in the DDL. Foreign keys use ON DELETE CASCADE, so removing a company, student, posting or application also removes the rows that depend on it [10]. Default values may be constants or expressions such as the current date [11], and ENUM columns limit a status to a fixed list of values [12].

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
students	Primary Key	student_id	Every student is uniquely identified.
	Unique, Not Null	email	No two students share an email address.
	Not Null	first_name, last_name	Name must be supplied.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
	Default	status = 'Unplaced'	A new student starts as unplaced.
	Check	cgpa between 0 and 10	Invalid CGPA values are rejected.
companies	Primary Key	company_id (auto-increment)	Every company is uniquely identified.
	Unique, Not Null	hr_email	One company record per HR email.
	Not Null	company_name	Company name must be supplied.
job_postings	Primary Key	job_id (auto-increment)	Every posting is uniquely identified.
	Foreign Key	company_id → companies	A posting belongs to a company; ON DELETE CASCADE.
	Not Null	job_title	Role title must be supplied.
applications	Primary Key	application_id (auto-increment)	Every application is uniquely identified.
	Foreign Key	student_id → students; job_id → job_postings	Application refers to an existing student and posting; ON DELETE CASCADE.
	Default	application_date = current date; status = 'Applied'	Date and status are set automatically.
interviews	Primary Key	interview_id (auto-increment)	Every interview round is uniquely identified.
	Foreign Key	application_id → applications	Interview belongs to an existing application; ON DELETE CASCADE.
	Default	round_number = 1; result = 'Pending'	Round and result get default values.
	Trigger	update_student_status (AFTER UPDATE on interviews)	When result becomes Selected, the student of that application is set to Placed.

The trigger is created by `placement.sql` and, if it is missing, by the function `init_db_schema()` when the application starts [13]. It fires when an existing interview row is updated. For an interview that is inserted with the result Selected, the application issues the same update on `students` explicitly. The view `placement_summary` joins five tables and lists only interviews whose result is Selected [14].

2.3 Functional Dependencies

A functional dependency $X \rightarrow Y$ states that the value of X determines the value of Y [15]. The important functional dependencies of the design are listed below. In each relation the determinant is a candidate key.

- `students`: $student_id \rightarrow first_name, last_name, email, phone, department, cgpa, status$; $email \rightarrow student_id$.

- **companies:** company_id → company_name, hr_email, industry, website; hr_email → company_id.
- **job_postings:** job_id → company_id, job_title, package_lpa, min_cgpa_required, deadline.
- **applications:** application_id → student_id, job_id, application_date, status.
- **interviews:** interview_id → application_id, interview_date, interviewer_name, round_number, result.

Business rules such as "a student applies to a posting at most once" and "a round number occurs once per application" are intended. The first is enforced only by a check in the application, and the second is not enforced; neither is part of the current DDL (see Section 4.1).

2.4 Normalization up to Third Normal Form (3NF)

Normalization, introduced by Codd [16] and explained for practical design by Kent [17], reduces redundancy and update anomalies [4], [18]. A single flat placement sheet would repeat the company name, HR email and package for every application, and the student's name and CGPA for every interview. Splitting it into the five relations above removes this repetition: each fact is stored once and linked through keys. Table 7 summarizes the normal-form status. All attributes are atomic (names are split into first and last name), every relation has a single-attribute key so no partial dependency can exist, and no non-key attribute depends on another non-key attribute.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
students	Yes	Yes	Yes	Yes	Candidate keys: student_id, email. The status column is a stored attribute that is kept consistent by the trigger and the application.
companies	Yes	Yes	Yes	Yes	Candidate keys: company_id, hr_email. Company data is stored once.
job_postings	Yes	Yes	Yes	Yes	Company details are not repeated; only company_id is stored.
applications	Yes	Yes	Yes	Yes	Resolves the many-to-many link between students and postings.
interviews	Yes	Yes	Yes	Yes	Multiple rounds per application are stored as separate rows, not repeating columns.

2.5 Data Dictionary

Table 8 gives the data dictionary for all tables.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
students	student_id	VARCHAR(20)	PK	Unique student identifier, e.g. STU101
	first_name	VARCHAR(50)	NOT NULL	First name of the student
	last_name	VARCHAR(50)	NOT NULL	Last name of the student
	email	VARCHAR(100)	UNIQUE, NOT NULL	Email address of the student
	phone	VARCHAR(15)	Optional	Contact number
	department	VARCHAR(50)	Optional	Department of the student
	cgpa	DECIMAL(4,2)	CHECK 0 to 10	Cumulative grade point average
	status	ENUM	DEFAULT 'Unplaced'	Unplaced or Placed
companies	company_id	INT	PK, AUTO_INCREMENT	Unique company identifier
	company_name	VARCHAR(100)	NOT NULL	Name of the company
	hr_email	VARCHAR(100)	UNIQUE, NOT NULL	Email of the HR contact
	industry	VARCHAR(50)	Optional	Industry sector, e.g. IT, Finance
	website	VARCHAR(255)	Optional	Company website
job_postings	job_id	INT	PK, AUTO_INCREMENT	Unique job posting identifier
	company_id	INT	FK → companies	Company offering the job
	job_title	VARCHAR(100)	NOT NULL	Role offered
	package_lpa	DECIMAL(5,2)	Optional	Annual package in LPA
	min_cgpa_required	DECIMAL(4,2)	Optional	Minimum CGPA for eligibility
	deadline	DATE	Optional	Last date to apply
applications	application_id	INT	PK, AUTO_INCREMENT	Unique application identifier
	student_id	VARCHAR(20)	FK → students	Applicant
	job_id	INT	FK → job_postings	Job applied for
	application_date	DATE	DEFAULT current date	Date of application

Table	Attribute	Data Type	Key / Constraint	Description
	status	ENUM	DEFAULT 'Applied'	Applied, Shortlisted or Rejected
interviews	interview_id	INT	PK, AUTO_INCREMENT	Unique interview identifier
	application_id	INT	FK → applications	Application being interviewed
	interview_date	DATETIME	Optional	Date and time of the round
	interviewer_name	VARCHAR(100)	Optional	Name of the interviewer
	round_number	INT	DEFAULT 1	Round sequence number
	result	ENUM	DEFAULT 'Pending'	Pending, Selected or Rejected

The view `placement_summary` has no stored data. It exposes `first_name`, `last_name` and `department` from `students`, `company_name` from `companies` and `package_lpa` from `job_postings` for every interview whose result is Selected.

2.6 Implementation of Logical Design: DDL and Sample Data

The design was implemented in MySQL. The DDL follows standard SQL [19], whose origins lie in the SEQUEL language [20], with MySQL-specific types such as ENUM. Table 9 gives the implementation environment. The CREATE TABLE statements for `students` and `applications`, and the trigger and view definitions, are shown below as representative DDL; the complete script is in Appendix A.

Table 9. Database Implementation Environment

Item	Details
DBMS	MySQL
DBMS Version	8.0.16 or later required for enforced CHECK constraints
Number of Tables	5 tables, 1 view and 1 trigger
Sample Data	8 students, 5 companies, 4 job postings, 2 applications and 2 interviews, loaded by the demo-data function of the application (SQL equivalent in Appendix A.2)

```
CREATE TABLE students (
  student_id VARCHAR(20) PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone VARCHAR(15),
  department VARCHAR(50),
  cgpa DECIMAL(4,2) CHECK (cgpa >= 0 AND cgpa <= 10.0),
  status ENUM('Unplaced', 'Placed') DEFAULT 'Unplaced'
);
```

```

CREATE TABLE applications (
    application_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id VARCHAR(20),
    job_id INT,
    application_date DATE DEFAULT (CURRENT_DATE),
    status ENUM('Applied','Shortlisted','Rejected') DEFAULT 'Applied',
    FOREIGN KEY (student_id) REFERENCES students(student_id)
        ON DELETE CASCADE,
    FOREIGN KEY (job_id) REFERENCES job_postings(job_id)
        ON DELETE CASCADE
);

CREATE TRIGGER update_student_status
AFTER UPDATE ON interviews
FOR EACH ROW
BEGIN
    IF NEW.result = 'Selected' THEN
        UPDATE students SET status = 'Placed'
        WHERE student_id = (SELECT student_id FROM applications
                           WHERE application_id = NEW.application_id);
    END IF;
END;

CREATE VIEW placement_summary AS
SELECT s.first_name, s.last_name, s.department,
       c.company_name, j.package_lpa
FROM interviews i
JOIN applications a ON i.application_id = a.application_id
JOIN students s ON a.student_id = s.student_id
JOIN job_postings j ON a.job_id = j.job_id
JOIN companies c ON j.company_id = c.company_id
WHERE i.result = 'Selected';

```

Representative sample records from the demo data are shown below.

```

INSERT INTO students (student_id, first_name, last_name, email, phone,
                     department, cgpa, status) VALUES
('STU101','Aarav','Sharma','aarav.sharma@woxsen.edu.in','9876501234',
 'AIML', 9.40, 'Unplaced'),
('STU102','Priya','Patel','priya.patel@woxsen.edu.in','9876501235',
 'CSE', 8.85, 'Unplaced');

INSERT INTO companies (company_name, hr_email, industry, website) VALUES
('Google India','careers@google.com','Big Tech / Software',
 'https://careers.google.com');

INSERT INTO job_postings (company_id, job_title, package_lpa,
                         min_cgpa_required, deadline) VALUES
(1, 'AI Research Associate', 38.00, 8.50, '2026-12-15');

```

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The application is a web portal written in Python with the Streamlit framework [21] and operated by the placement cell administrator. The file `app.py` contains the user interface and all database operations, and `db.py` provides the connection settings and creates the trigger and view if they are missing. A sidebar menu opens eight pages: an executive dashboard, management pages for students, companies, job postings, applications and interviews, a reports page, and a database page for diagnostics and demo data. Query results are handled as pandas data frames [22] and the dashboard charts are drawn with Plotly [23]. Every page reads from or writes to the MySQL database `campus_placement`; no static data are shown, and the optional demo data are also inserted into the database. Selection lists built from the database let the administrator choose existing students, companies, jobs and applications, so that valid foreign key values are always used. A simpler console version of the system (`main.py`) is retained for reference and supports adding and viewing records only.

3.2 Development Environment and Database Connectivity

Table 10 lists the development environment. The application connects to MySQL using MySQL Connector/Python, a driver that follows the Python Database API specification [24] and is described in the vendor guide [25].

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3
DBMS	MySQL
IDE / Framework / GUI Toolkit	VS Code; Streamlit (web UI) with pandas and Plotly
Database Connector / Driver	mysql-connector-python (<code>mysql.connector</code>)
Operating Environment	Windows; local MySQL server on localhost (port 3306)
Repository Link	https://github.com/Farhan8012/DBMS-PBL.git

The function `get_db_connection()` in `db.py` builds the connection from the environment variables `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` and `DB_PORT`, with local defaults, and enables autocommit. The sidebar calls `test_db_connection()` to show the MySQL online or offline badge, and `init_db_schema()` creates the trigger

and view if they are absent. In `app.py`, the helper `run_query()` executes a `SELECT` with parameters and returns a pandas data frame, and `execute_dml()` executes `INSERT`, `UPDATE` and `DELETE` statements and returns success or the MySQL error message. Each helper opens a connection, runs one parameterized statement and closes the connection. The application is started with `streamlit run app.py`. The credentials are shown as a placeholder below.

```
DEFAULT_CONFIG = {
    "host": os.getenv("DB_HOST", "localhost"),
    "user": os.getenv("DB_USER", "root"),
    "password": os.getenv("DB_PASSWORD", "<password>"),
    "database": os.getenv("DB_NAME", "campus_placement"),
    "port": int(os.getenv("DB_PORT", 3306))
}

def get_db_connection(host=None, user=None, password=None,
                      database=None, port=None):
    try:
        connection = mysql.connector.connect(
            host=host or DEFAULT_CONFIG["host"],
            user=user or DEFAULT_CONFIG["user"],
            password=password if password is not None
                else DEFAULT_CONFIG["password"],
            database=database or DEFAULT_CONFIG["database"],
            port=port or DEFAULT_CONFIG["port"],
            autocommit=True)
        return connection
    except Error as err:
        print(f"Database Connection Error: {err}")
        return None
```

3.3 Application Modules and Database Mapping

Table 11 maps each application module to the tables it uses and the main database operations it performs.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Connection layer (<code>db.py</code>)	Connects to MySQL, reports status and ensures the trigger and view exist	All tables; trigger; view	Connect; SHOW TRIGGERS; CREATE TRIGGER; CREATE OR REPLACE VIEW
Executive Dashboard	KPI cards, four charts and the placed-candidates list	All five tables; placement_summary	SELECT with scalar subqueries, JOIN and GROUP BY
Students Directory	Register, search, edit and remove students	students	INSERT; SELECT with LIKE filters; UPDATE; DELETE
Partner Companies	Register, search, edit and remove companies	companies, job_postings	INSERT; SELECT with LEFT JOIN and COUNT; UPDATE; DELETE

Module / Screen	Purpose	Tables Used	Main Operations
Job Postings & Drives	Post, filter, edit and remove jobs	job_postings, companies, applications	INSERT; SELECT with JOIN and LEFT JOIN; UPDATE; DELETE
Applications Pipeline	Submit with eligibility and duplicate checks, update status, withdraw	applications, students, job_postings, companies	INSERT; SELECT with four-table JOIN; UPDATE; DELETE
Interview Management	Schedule rounds, update results, cancel	interviews, applications, students, job_postings, companies	INSERT; SELECT with five-table JOIN; UPDATE (fires trigger); DELETE
Advanced Reports & Views	Master report, view output and SQL explorer	All five tables; placement_summary	Five-table JOIN; SELECT on view; read-only SELECT
Database & Demo Data	Connection diagnostics and demo data seeding	All five tables	INSERT guarded by existence checks; UPDATE

3.4 CRUD Operations

Table 12 shows how the four CRUD operations are supported by the application.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Register student, register company, post job, submit application and schedule interview, using forms and INSERT with parameters	All five tables	Figures 3 and 5; TC-01, TC-06
Read / Search	Directory and list pages with search and filters; dashboard; reports using SELECT and JOIN; CSV download	All five tables	Figures 2, 4 and 7; TC-01
Update	Edit student, company and job details; update application status; update interview result	All five tables	Figure 6; TC-08, TC-11
Delete / Controlled Removal	Remove student, company or job; withdraw application; cancel interview, each with a warning and a confirm button; dependent rows removed by ON DELETE CASCADE	All five tables	TC-09

3.5 Business Rules, Validation and Error Handling

The application validates input in layers. Streamlit widgets restrict numeric entry to valid ranges and the Python code checks required fields and the email format. The database then rejects values that break keys or constraints, and the MySQL error returned by `execute_dml()` is shown to the administrator. All data statements use parameter placeholders (`%s`), so user input is not concatenated into a query, which protects against SQL injection [26]. The only exception is the SQL explorer, which runs the administrator's text as typed after checking that it begins with `SELECT`, `SHOW` or `DESCRIBE`; this is a convenience

check and not a full security control. Exceptions are handled with the standard Python try and except model [27]. Destructive pages show a warning about cascading deletion and require a confirm button. The connection uses autocommit, so each statement is committed as soon as it succeeds, which makes a single statement atomic [28], [29]; multi-step actions are not wrapped in one transaction (Section 4.1). Table 13 lists the main rules.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Required fields and email format	Python checks for empty fields and a regular expression for the email before INSERT	"Please fill in all required fields marked with *." and "Please enter a valid email address." (TC-02, TC-03)
CGPA between 0 and 10; package and cut-off in range	number_input widgets with minimum and maximum values; CHECK constraint on cgpa	Widget limits input; error 3819 from MySQL for a direct insert (TC-05)
Unique student ID, student email and HR email	PRIMARY KEY and UNIQUE constraints; MySQL error shown by the application	"Failed to add student: 1062 ... Duplicate entry" (TC-04)
Application needs an existing student and job	Selection lists from the database; FOREIGN KEY constraints; warning if a list is empty	"Need both registered students and active job postings to submit applications."
Eligibility check	Candidate CGPA compared with min_cgpa_required; advisory only	Passed or Warning message; submission still allowed (TC-06; Figure 5)
No duplicate application	SELECT on student_id and job_id before INSERT; submit blocked if a row exists	"This student has already submitted an application for this role" (TC-07)
Selected result marks student Placed	Trigger update_student_status plus an explicit UPDATE in the application	Success message and status Placed in directory and view (TC-08; Figure 6)
Controlled deletion	Warning text, confirm button and ON DELETE CASCADE	"Student removed successfully"; dependent rows disappear (TC-09)
Read-only SQL explorer	Statement must begin with SELECT, SHOW or DESCRIBE	"Only read-only queries (SELECT, SHOW, DESCRIBE) are permitted" (TC-10)

3.6 Reports and Database Operations

Table 14 lists the reports and the SQL concepts used. Inner joins combine rows of related tables on matching key values [30]. The five-table master report is the most complete and its query is shown below.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Dashboard KPIs	Scalar subqueries with COUNT, MAX and AVG	Cards: total students, placement rate, highest and average package, companies, jobs, applications, interview rounds

Report / Operation	SQL / Database Concept Used	Application Output
Package by company and role	JOIN of job_postings and companies with ORDER BY	Horizontal bar chart
Department placement distribution	GROUP BY with SUM(CASE WHEN ...)	Stacked bar chart of placed and unplaced students
Application and interview status	GROUP BY with COUNT	Two donut charts
Company directory and job list	LEFT JOIN with COUNT and GROUP BY	Tables with jobs posted and applicants per job
Applications report	Four-table INNER JOIN	Table with student, company, role, status, date
Five-table master report	Five-table INNER JOIN with CONCAT	Table with CSV export
placement_summary view	View over a five-table JOIN with WHERE result = Selected	Table and metrics: placed candidates, total and average package
SQL explorer presets	ORDER BY with LIMIT; GROUP BY with CASE; filtering; LEFT JOIN with GROUP BY	Result tables with row count and execution time

```

SELECT i.interview_id, s.student_id,
       CONCAT(s.first_name, ' ', s.last_name) AS student_name,
       s.department, s.cgpa, c.company_name, c.industry,
       j.job_title, j.package_lpa, a.application_date,
       i.round_number, i.interview_date, i.interviewer_name,
       i.result AS interview_result, s.status AS student_status
FROM interviews i
JOIN applications a ON i.application_id = a.application_id
JOIN students s ON a.student_id = s.student_id
JOIN job_postings j ON a.job_id = j.job_id
JOIN companies c ON j.company_id = c.company_id
ORDER BY i.interview_date DESC;

```

3.7 Application Screens

Figures 2 to 7 show the application in use: the dashboard (Figure 2), data entry (Figure 3), search and display (Figure 4), a transaction that links a student to a job with the eligibility check (Figure 5), an update that triggers placement (Figure 6) and the reports (Figure 7).

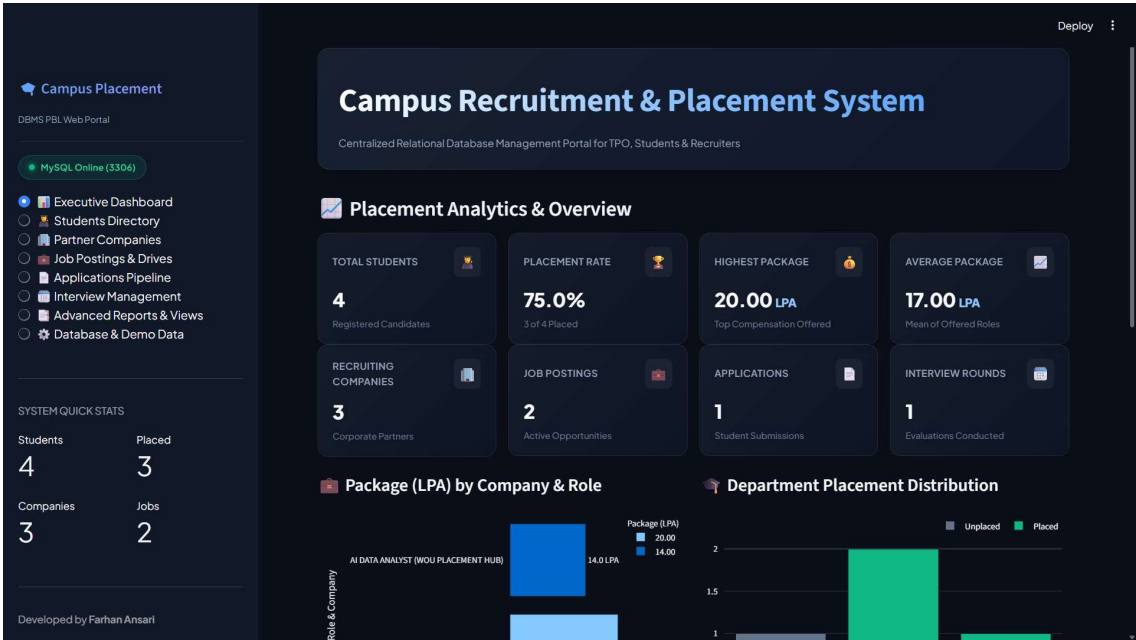


Figure 2. Executive Dashboard

The screenshot shows the 'Students Management' section of the system, specifically the 'Register a New Student' form. The form includes fields for Student ID (3457), Last Name (Royal), First Name (None), Phone Number (917463746534), Email (nene12@gmail.com), CGPA (8.50), Department (CSE), and Initial Status (Placed). A 'Register Student' button is at the bottom. The left sidebar is identical to Figure 2. The main content area also includes links for 'Student Directory', 'Register New Student', 'Edit Student', and 'Remove Student'. The dashboard is developed by Farhan Ansari.

Figure 3. Student Registration Form

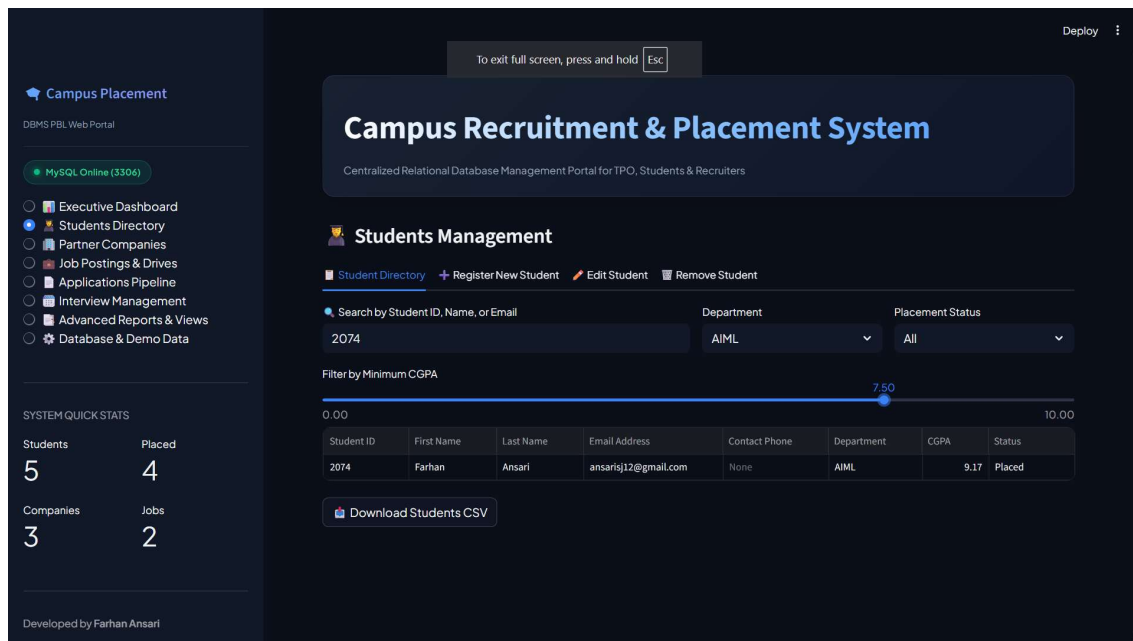


Figure 4. Student Directory with Search and Filters

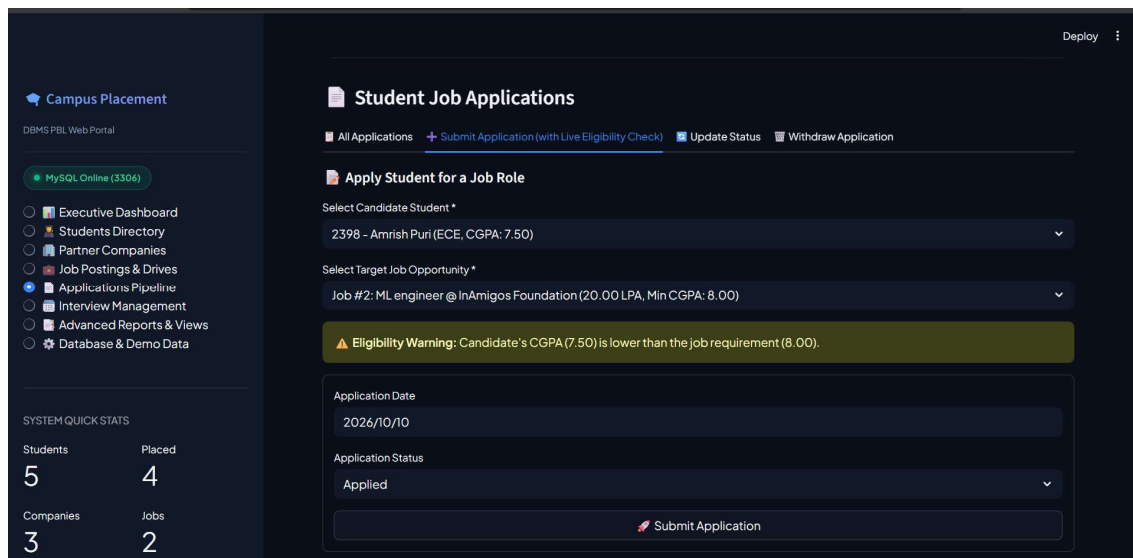


Figure 5. Application Submission with Live Eligibility Check

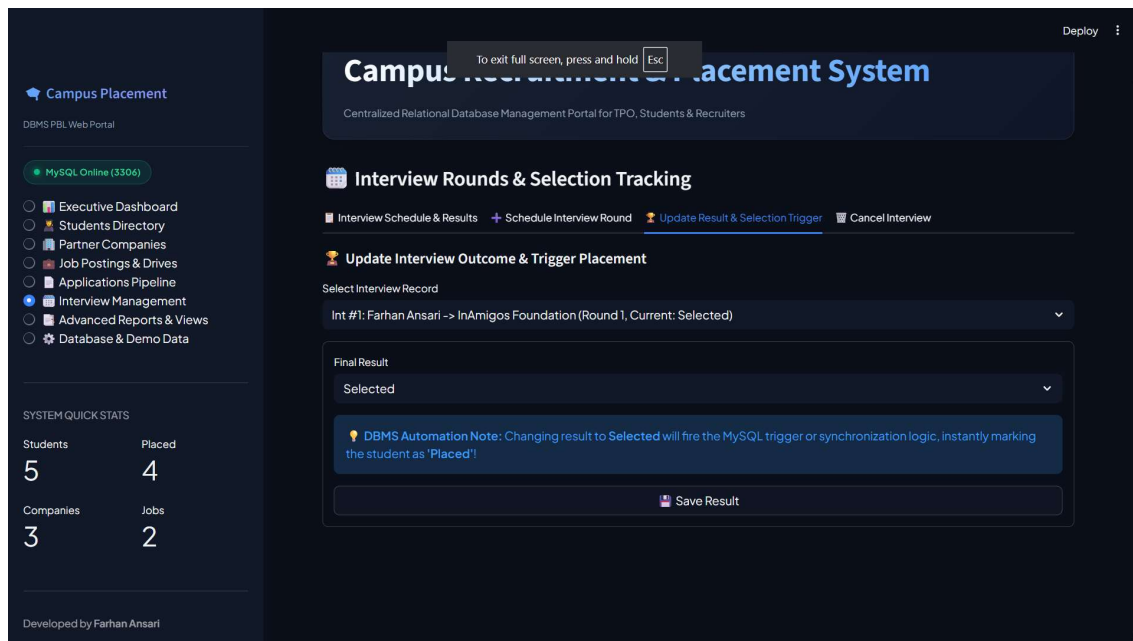


Figure 6. Interview Result Update Triggering Placement



Figure 7. Five-Table Master Report and placement_summary View

3.8 Testing and Results

Table 15 lists the test cases, covering positive and negative scenarios. They use the demo data of Section 2.6. The expected results follow from the schema and application code. The shaded

Actual Result and Status cells are to be confirmed against the observed output when the tests are run.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Register student (positive)	STU110, Neha Singh, valid email, AIML, CGPA 8.50	Success message; student appears in the directory with status Unplaced	Success message shown; record listed with status Unplaced	Pass
TC-02	Required field (negative)	Register a student with an empty first name	Error "Please fill in all required fields marked with *."; no record added	Error shown; no record added	Pass
TC-03	Email format (negative)	Email entered as "neha.singh"	Error "Please enter a valid email address."; no record added	Error shown; no record added	Pass
TC-04	Duplicate ID (negative)	Register STU101 again	Error with MySQL 1062 duplicate entry; no record added	Error 1062 duplicate entry for key students.PRIMARY	Pass
TC-05	CHECK constraint (negative)	In MySQL client: INSERT a student with CGPA 11	Error 3819, check constraint violated; no row inserted	Error 3819 check constraint violated; no row inserted	Pass
TC-06	Eligibility warning	Select STU103 (CGPA 7.90) and the Google AI Research Associate job (minimum 8.50)	Warning shows both CGPA values; submission is still allowed	Warning shown with 7.90 and 8.50; submit button available	Pass
TC-07	Duplicate application (negative)	Submit STU101 for the same job again	Error with existing application ID; submission blocked	Error shown with application ID 1; no new row	Pass
TC-08	Trigger and placement (positive)	Submit STU102 for the Amazon Cloud Support Engineer job, schedule round 1 as Pending, then set the result to Selected	Success message; STU102 status becomes Placed; row appears in placement_summary	Success message shown; status Placed; row in view	Pass
TC-09	Cascade delete	Remove student STU108	Student removed; the student's application and interview no longer appear	Student removed; related rows no longer listed	Pass
TC-10	Read-only explorer (negative)	Run "DELETE FROM students" in the SQL explorer	Error "Only read-only queries ... are permitted"; data unchanged	Error shown; data unchanged	Pass
TC-11	Update student	Edit STU101 and change the phone number	Success message; new phone shown in the directory	Success message shown; new phone listed	Pass

Key results. The positive cases confirm that records entered through the portal are stored in MySQL, changed by update statements, and read back through the join queries and the view.

The negative cases confirm that invalid values are stopped by the Python checks or by the database constraints, that a duplicate application is refused by the application, and that a Selected result marks the student as placed. The application therefore uses the designed database and enforces the key integrity rules. Three limits remain: the eligibility check is advisory and does not block an application, the duplicate rule is not backed by a UNIQUE constraint, and the trigger fires on updates only. These are listed as future enhancements.

CHAPTER 4

CONCLUSION

This project addressed the problem of managing campus placement data that is otherwise spread across spreadsheets and separate records. An entity-relationship model with five entities was converted into five normalized relations, which were implemented in MySQL with primary key, foreign key, unique, check and default constraints, a trigger that marks selected students as placed, and a view that summarizes placements. The schema is in third normal form and also satisfies BCNF.

A Python web application was developed with Streamlit on top of this schema. It performs create, read, update and delete operations on all entities through parameterized queries, checks eligibility and duplicate applications, shows an analytics dashboard, produces join-based reports and provides a read-only SQL explorer. The project gave practical experience of moving from ER modelling and normalization to SQL, triggers and views, and a working database application.

4.1 Limitations and Future Enhancements

- The eligibility check only warns the administrator. Applications from students below the minimum CGPA, or after the deadline, should be blocked by the application and by a database rule (FR-13).
- The duplicate-application check is made in the application only. UNIQUE constraints on (student_id, job_id) and on (application_id, round_number) should be added to the DDL.
- The trigger fires only when an interview row is updated, so the application repeats the update for new rows. A student is not returned to Unplaced if a Selected result is changed or the interview is deleted; an AFTER INSERT trigger and a recalculation of the status would fix this.
- There is no login or role-based access, and db.py contains a default password that should be removed so that credentials come only from environment variables. The SQL explorer relies on a prefix check, and a read-only MySQL account should be used instead.

- Each query opens its own connection with autocommit, and multi-step actions such as inserting an interview and updating the student are not in one transaction. Connection pooling and explicit transactions with rollback would improve reliability.
- The separate offer, skill and placement drive entities from the Review-I model, student and recruiter portals, notifications, and indexes for larger data volumes can be added as the system grows.

REFERENCES

- [1] A. Mishra, A. Yadav, and P. Raj, "Campus recruitment system," EasyChair Preprint 5909, Jun. 2021. Accessed: Oct. 10, 2026. [Online]. Available: <https://easychair.org/publications/preprint/F1T5>
- [2] J. Shah, "Placement system development," M.Tech. thesis, Dhirubhai Ambani Inst. Inf. Commun. Technol. (DA-IICT), Gandhinagar, India, 2019.
- [3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
- [4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [5] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. New York, NY, USA: McGraw-Hill, 2003.
- [6] Oracle Corporation, "CHECK constraints," *MySQL 8.0 Reference Manual*. Accessed: Oct. 10, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html>
- [7] P. P.-S. Chen, "The entity-relationship model—toward a unified view of data," *ACM Trans. Database Syst.*, vol. 1, no. 1, pp. 9–36, Mar. 1976, doi: 10.1145/320434.320440.
- [8] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.
- [9] T. M. Connolly and C. E. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Harlow, U.K.: Pearson, 2015.
- [10] Oracle Corporation, "FOREIGN KEY constraints," *MySQL 8.0 Reference Manual*. Accessed: Oct. 10, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>
- [11] Oracle Corporation, "Data type default values," *MySQL 8.0 Reference Manual*. Accessed: Oct. 10, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/data-type-defaults.html>
- [12] Oracle Corporation, "The ENUM type," *MySQL 8.0 Reference Manual*. Accessed: Oct. 10, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/enum.html>

- [13] Oracle Corporation, "CREATE TRIGGER statement," *MySQL 8.0 Reference Manual*. Accessed: Oct. 10, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/create-trigger.html>
- [14] Oracle Corporation, "CREATE VIEW statement," *MySQL 8.0 Reference Manual*. Accessed: Oct. 10, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/create-view.html>
- [15] H. Garcia-Molina, J. D. Ullman, and J. Widom, *Database Systems: The Complete Book*, 2nd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2009.
- [16] E. F. Codd, "Further normalization of the data base relational model," in *Data Base Systems*, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.
- [17] W. Kent, "A simple guide to five normal forms in relational database theory," *Commun. ACM*, vol. 26, no. 2, pp. 120–125, Feb. 1983, doi: 10.1145/358024.358054.
- [18] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.
- [19] *Information Technology—Database Languages—SQL—Part 2: Foundation (SQL/Foundation)*, ISO/IEC 9075-2:2016, 2016.
- [20] D. D. Chamberlin and R. F. Boyce, "SEQUEL: A structured English query language," in *Proc. ACM SIGFIDET Workshop Data Description, Access and Control*, Ann Arbor, MI, USA, 1974, pp. 249–264, doi: 10.1145/800296.811515.
- [21] Streamlit Inc., *Streamlit Documentation*. Accessed: Oct. 10, 2026. [Online]. Available: <https://docs.streamlit.io/>
- [22] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, Austin, TX, USA, 2010, pp. 56–61, doi: 10.25080/Majora-92bf1922-00a.
- [23] Plotly Technologies Inc., *Plotly Python Graphing Library Documentation*. Accessed: Oct. 10, 2026. [Online]. Available: <https://plotly.com/python/>
- [24] M.-A. Lemburg, "PEP 249 – Python Database API specification v2.0," Python Software Foundation. Accessed: Oct. 10, 2026. [Online]. Available: <https://peps.python.org/pep-0249/>
- [25] Oracle Corporation, *MySQL Connector/Python Developer Guide*. Accessed: Oct. 10, 2026. [Online]. Available: <https://dev.mysql.com/doc/connector-python/en/>

- [26] OWASP Foundation, "SQL injection prevention cheat sheet," *OWASP Cheat Sheet Series*. Accessed: Oct. 10, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [27] Python Software Foundation, "Errors and exceptions," *The Python Tutorial*. Accessed: Oct. 10, 2026. [Online]. Available: <https://docs.python.org/3/tutorial/errors.html>
- [28] J. Gray, "The transaction concept: Virtues and limitations," in *Proc. 7th Int. Conf. Very Large Data Bases (VLDB)*, Cannes, France, 1981, pp. 144–154.
- [29] T. Haerder and A. Reuter, "Principles of transaction-oriented database recovery," *ACM Comput. Surv.*, vol. 15, no. 4, pp. 287–317, Dec. 1983, doi: 10.1145/289.291.
- [30] Oracle Corporation, "JOIN clause," *MySQL 8.0 Reference Manual*. Accessed: Oct. 10, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/join.html>

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete DDL, trigger and view are in `placement.sql`, and the demo data are inserted by the application; their SQL equivalent is in `placement_sample_data.sql`. Both scripts are reproduced below; long lines are wrapped for printing.

A.1 Schema script (`placement.sql`)

```
USE campus_placement;

-- Table 1: Students
CREATE TABLE students (
    student_id VARCHAR(20) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    department VARCHAR(50),
    cgpa DECIMAL(4,2) CHECK (cgpa >= 0 AND cgpa <= 10.0),
    status ENUM('Unplaced', 'Placed') DEFAULT 'Unplaced'
);

-- Table 2: Companies
CREATE TABLE companies (
    company_id INT AUTO_INCREMENT PRIMARY KEY,
    company_name VARCHAR(100) NOT NULL,
    hr_email VARCHAR(100) UNIQUE NOT NULL,
    industry VARCHAR(50),
    website VARCHAR(255)
);

-- Table 3: Job Postings
CREATE TABLE job_postings (
    job_id INT AUTO_INCREMENT PRIMARY KEY,
    company_id INT,
    job_title VARCHAR(100) NOT NULL,
    package_lpa DECIMAL(5,2),
    min_cgpa_required DECIMAL(4,2),
    deadline DATE,
    FOREIGN KEY (company_id) REFERENCES companies(company_id) ON DELETE
        CASCADE
);

-- Table 4: Applications
-- This links Students to the Jobs they apply for
CREATE TABLE applications (
    application_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id VARCHAR(20),
    job_id INT,
    application_date DATE DEFAULT (CURRENT_DATE),
    status ENUM('Applied', 'Shortlisted', 'Rejected') DEFAULT 'Applied',
    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE
        CASCADE,
    FOREIGN KEY (job_id) REFERENCES job_postings(job_id) ON DELETE CASCADE
);
```

```

-- Table 5: Interviews
-- This tracks the interview rounds for shortlisted applications
CREATE TABLE interviews (
    interview_id INT AUTO_INCREMENT PRIMARY KEY,
    application_id INT,
    interview_date DATETIME,
    interviewer_name VARCHAR(100),
    round_number INT DEFAULT 1,
    result ENUM('Pending', 'Selected', 'Rejected') DEFAULT 'Pending',
    FOREIGN KEY (application_id) REFERENCES applications(application_id)
    ON DELETE CASCADE
);

DELIMITER //
CREATE TRIGGER update_student_status
AFTER UPDATE ON interviews
FOR EACH ROW
BEGIN
    IF NEW.result = 'Selected' THEN
        UPDATE students
        SET status = 'Placed'
        WHERE student_id = (SELECT student_id FROM applications WHERE
            application_id = NEW.application_id);
    END IF;
END; //
DELIMITER ;

CREATE VIEW placement_summary AS
SELECT s.first_name, s.last_name, s.department, c.company_name,
       j.package_lpa
FROM interviews i
JOIN applications a ON i.application_id = a.application_id
JOIN students s ON a.student_id = s.student_id
JOIN job_postings j ON a.job_id = j.job_id
JOIN companies c ON j.company_id = c.company_id
WHERE i.result = 'Selected';

```

A.2 Demo data script (placement_sample_data.sql)

```

USE campus_placement;

-- SQL equivalent of the "Seed Sample Demo Records" button in app.py.
-- Run after placement.sql on an empty database
-- (the numeric IDs below assume auto-increment values start from 1).

INSERT INTO companies (company_name, hr_email, industry, website) VALUES
('Google India', 'careers@google.com', 'Big Tech / Software',
 'https://careers.google.com'),
('Microsoft IDC', 'campus@microsoft.com', 'Enterprise Cloud',
 'https://careers.microsoft.com'),
('Amazon AWS', 'aws-hiring@amazon.com', 'Cloud Infrastructure',
 'https://amazon.jobs'),
('Goldman Sachs', 'recruiting@gs.com', 'Fintech & Investment',
 'https://goldmansachs.com'),
('TCS Digital', 'campus@tcs.com', 'IT Consulting',
 'https://tcs.com');

INSERT INTO students (student_id, first_name, last_name, email, phone,
    department, cgpa, status) VALUES

```

```

('STU101', 'Aarav', 'Sharma', 'aarav.sharma@woxsen.edu.in', '9876501234',
 'AIML', 9.40, 'Unplaced'),
('STU102', 'Priya', 'Patel', 'priya.patel@woxsen.edu.in', '9876501235',
 'CSE', 8.85, 'Unplaced'),
('STU103', 'Rahul', 'Verma', 'rahul.verma@woxsen.edu.in', '9876501236',
 'ECE', 7.90, 'Unplaced'),
('STU104', 'Ananya', 'Reddy', 'ananya.reddy@woxsen.edu.in', '9876501237',
 'Data Science', 9.10, 'Unplaced'),
('STU105', 'Karan', 'Mehta', 'karan.mehta@woxsen.edu.in', '9876501238',
 'CSE', 8.20, 'Unplaced'),
('STU106', 'Sneha', 'Nair', 'sneha.nair@woxsen.edu.in', '9876501239',
 'AIML', 7.45, 'Unplaced'),
('STU107', 'Vikram', 'Iyer', 'vikram.iyer@woxsen.edu.in', '9876501240',
 'IT', 8.60, 'Unplaced'),
('STU108', 'Riya', 'Sen', 'riya.sen@woxsen.edu.in', '9876501241',
 'CSE', 9.75, 'Unplaced');

INSERT INTO job_postings (company_id, job_title, package_lpa,
 min_cgpa_required, deadline) VALUES
(1, 'AI Research Associate', 38.00, 8.50, '2026-12-15'),
(2, 'Software Engineer II', 32.00, 8.00, '2026-12-20'),
(3, 'Cloud Support Engineer', 24.00, 7.50, '2026-11-30'),
(4, 'Quantitative Analyst', 28.00, 8.00, '2026-12-10');

INSERT INTO applications (student_id, job_id, application_date, status)
VALUES
('STU101', 1, CURRENT_DATE, 'Shortlisted'),
('STU108', 2, CURRENT_DATE, 'Shortlisted');

INSERT INTO interviews (application_id, interview_date, interviewer_name,
 round_number, result) VALUES
(1, NOW(), 'Dr. Sundar / HR', 1, 'Selected'),
(2, NOW(), 'Satya / Lead Arch', 1, 'Selected');

-- The trigger fires only on UPDATE, so the application also sets the
status explicitly.
UPDATE students SET status = 'Placed' WHERE student_id IN ('STU101',
 'STU108');

```

APPENDIX B - APPLICATION SOURCE CODE

The application source consists of three Python files: `app.py` (the Streamlit web application with all pages and database operations), `db.py` (connection settings, connection test and creation of the trigger and view) and `main.py` (the earlier console version). The complete code is available at the repository given in Table 10: <https://github.com/Farhan8012/DBMS-PBL.git>. The code is not reproduced here to keep the report concise.

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

The following queries are the preset analytical queries of the SQL explorer page. Their output screenshots can be attached below each query.

C.1 Top five highest paying roles

```
SELECT c.company_name, j.job_title, j.package_lpa, j.min_cgpa_required
FROM job_postings j
JOIN companies c ON j.company_id = c.company_id
ORDER BY j.package_lpa DESC LIMIT 5;
```

C.2 Department placement statistics

```
SELECT department, COUNT(*) AS total_students,
      SUM(CASE WHEN status = 'Placed' THEN 1 ELSE 0 END) AS placed_count,
      ROUND(SUM(CASE WHEN status = 'Placed' THEN 1 ELSE 0 END)
            * 100.0 / COUNT(*), 1) AS placement_percentage
FROM students GROUP BY department;
```

C.3 Unplaced students with CGPA of 8.0 or more

```
SELECT student_id, first_name, last_name, department, cgpa, email
FROM students
WHERE status = 'Unplaced' AND cgpa >= 8.0
ORDER BY cgpa DESC;
```

C.4 Companies by total applications received

```
SELECT c.company_name, COUNT(a.application_id) AS total_applications
FROM companies c
JOIN job_postings j ON c.company_id = j.company_id
LEFT JOIN applications a ON j.job_id = a.job_id
GROUP BY c.company_id, c.company_name
ORDER BY total_applications DESC;
```